Jose Daniel Egea Cortacero

PRÁCTICA 2

Integración ETL

CURSO 25/26

Sistemas Multidimensionales

ÍNDICE

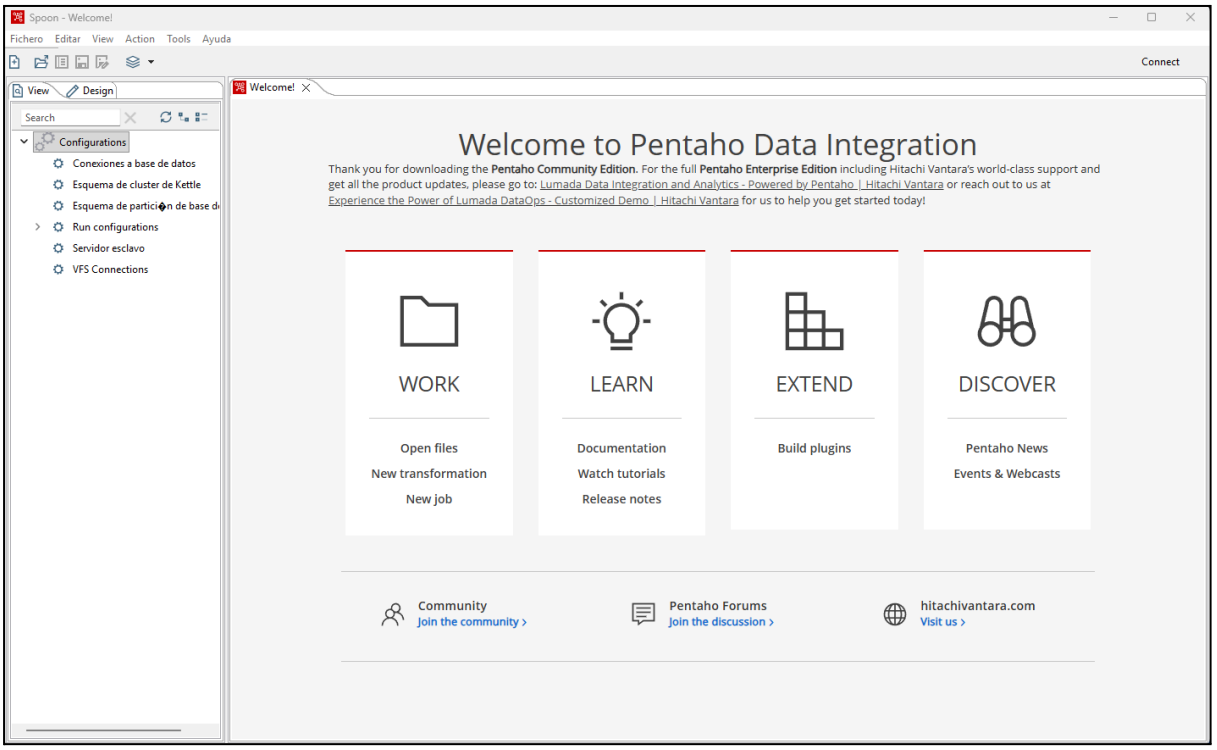
1. Preparación del entorno..... 2
2. Data Profiling..... 4
3. Diseño del componente ETL..... 5
3.1. Arquitectura ETL..... 5
3.2. Flujo de transformaciones..... 6
3.3. Resultados del flujo ETL..... 7
4. Implementación a nuestro sistema OLAP..... 8
4.1. Análisis de los datos actuales..... 9
4.2. Desarrollo del flujo..... 9
4.4. Visualización de resultados..... 18
4.5. Decisiones de diseño..... 19
5. Integración de objetivos en consulta analítica..... 20
5.1. Fichero CSV..... 20
5.2. Diseño del flujo..... 20
5.3. Configuración de los pasos..... 21
5.4. Resultados..... 24

Enlace a GitHub: https://github.com/DaniEgeaCortacero/SM_Sistema_OLAP

1. Preparación del entorno

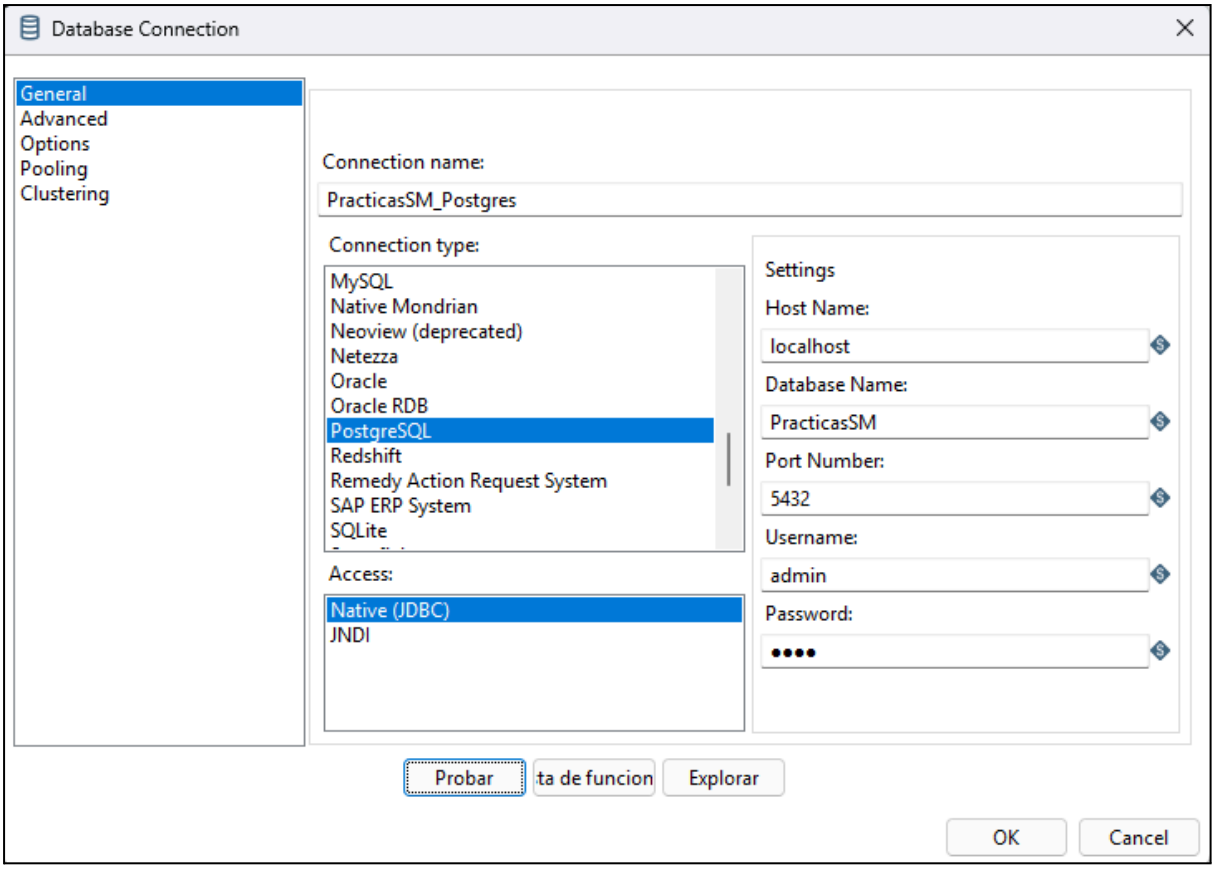
Para construir y gestionar los datos mediante ETL, necesitamos instalar, además de lo anteriormente mencionado durante la Práctica 1, el software **Pentaho Developer Edition** donde para ello necesitaremos también **Java SDK 17** y el **driver de PostgreSQL** para su integración.

Una vez descargado e instalado lo anterior, podremos ver la siguiente interfaz accediendo mediante **Spoon.bat** del directorio de la instalación de Pentaho:

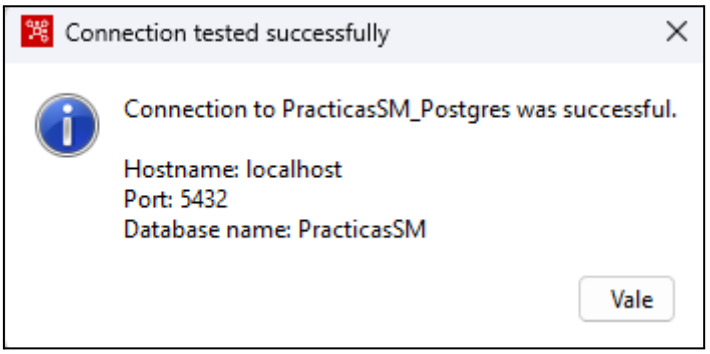


Para conectar con la base de datos, vamos a **"Conexiones a base de datos" > Nuevo e introducimos los datos.**

Nota: Si se usa el docker subido de GitHub, los datos son los siguiente:



Luego hacemos clic en “**Probar**” y debería aparecer lo siguiente:



2. Data Profiling

Conocer los datos que se almacenan en la empresa es vital para poder gestionarlos en futuras aplicaciones, por lo que tener los datos normalizados y consistentes nos ayuda a tener un control de los datos de la empresa con exactitud.
Sin embargo, a veces no están bien definidos (no son datos limpios) y se necesita hacer procesos ETL sobre estos datos.

Usando aplicaciones como Pentaho, podemos automatizar este proceso para dar solución a los datos mal definidos o registros que no aportan nada a la empresa debido a errores en esta (como puede ser a falta de claves foráneas o datos inválidos)

Ejecutando las consultas del fichero **02_profiling_queries.sql** se han obtenido los siguientes resultados y se han deducido las siguientes reglas de limpieza:

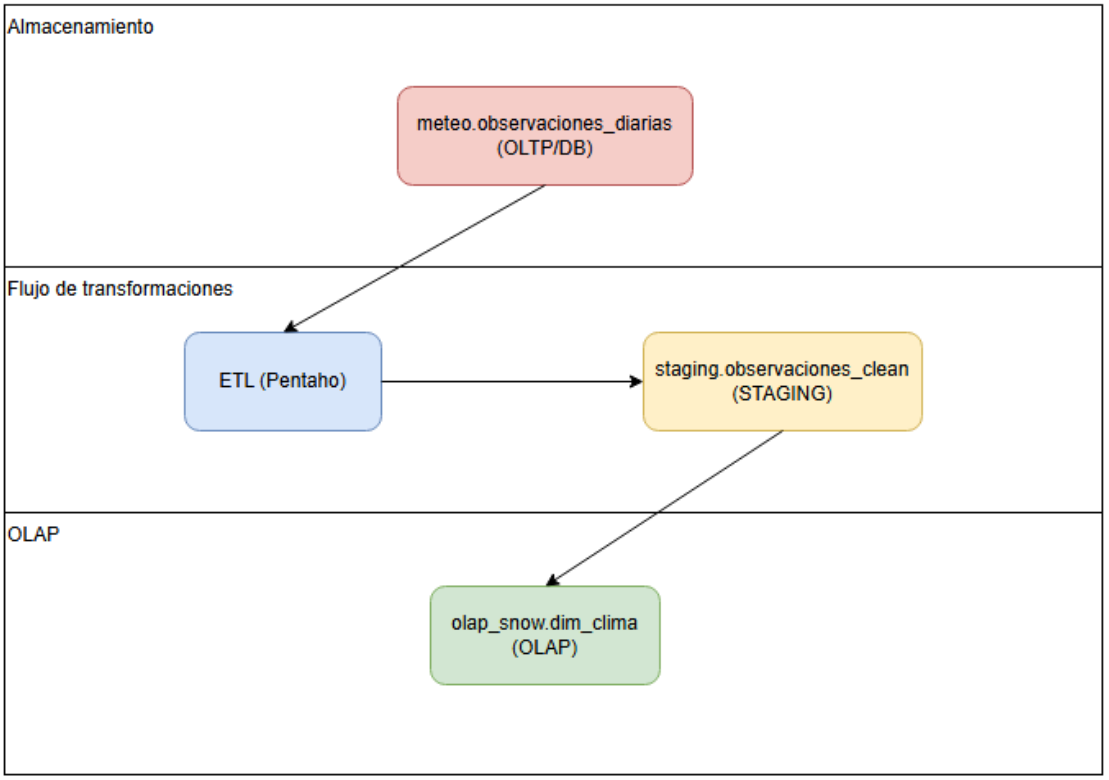
ID	Anomalía	Query	Registros afectados	Regla de limpieza
A1	Error de sensor (-99)	P03	15	temp = -99 → NULL
A2	Precipitación negativa	P04	8	precip < 0 → 0.0
A3	Humedad fuera de [0,100]	P05	5	humedad < 0 o > 100 -> NULL
A4	Ciudad inconsistente	P07	7	aplicar TRIM, normalizar mayus / minus
A5	Temp mínima > máxima	P06	16	temp_min > temp_max → intercambiar valores
A6	FK huérfana estacion_id	P08	12	excluir registro
A7	Fila completamente nula	P02	1 ⇒ (0, 0.02)	excluir registro
A8	Duplicado (fecha, ciudad)	P09	1 ⇒ (12)	borrar duplicados
A9	Vocabulario inválido	P10	15	excluir / estandarizar nombre

3. Diseño del componente ETL

3.1. Arquitectura ETL

Disponemos de 3 capas en nuestra base de datos:

- **Meteo:** es el origen de nuestros datos, aquí se almacenan todo tipo de datos, ya sean correctos o no.
- **Staging:** es nuestro área de limpieza y transformación de datos. Se almacenan los datos de los distintos pasos del flujo ETL.
- **OLAP:** se almacenan los datos cargados desde el último paso de nuestro flujo ETL, cuando los datos a están limpios y listos para ser usados para su análisis.



3.2. Flujo de transformaciones

El flujo de transformaciones permite procesar los datos desde su origen hasta la carga final en el Data Warehouse. Incluye capas de limpieza, validación, enriquecimiento y carga de datos.

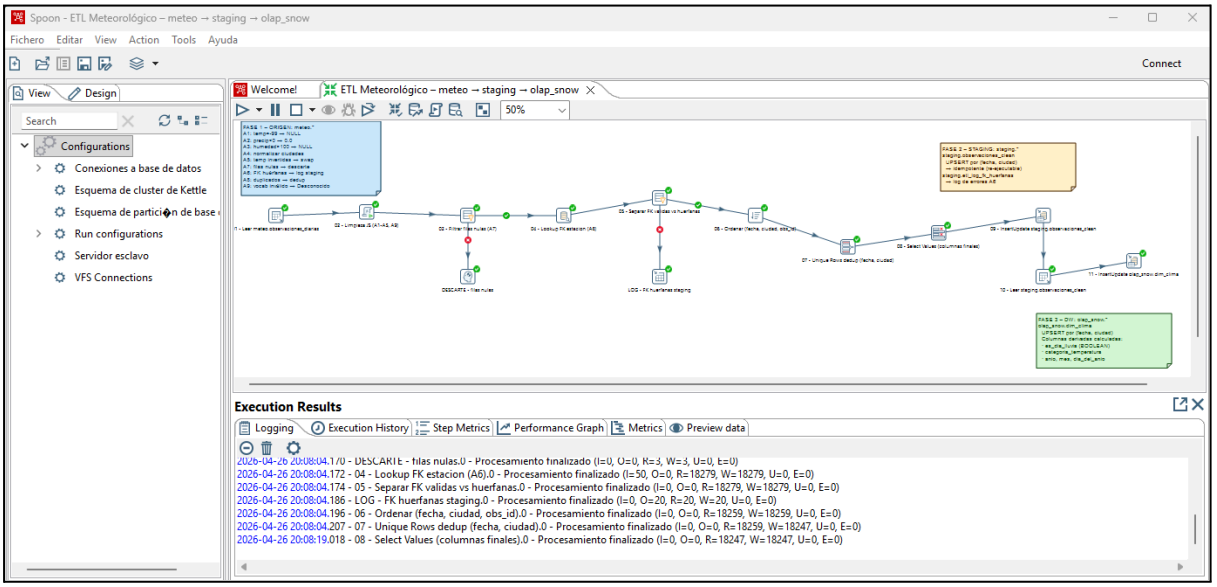
En esta práctica, se ha realizado el siguiente flujo:

1. **Leer meteo.observaciones_diarias:** se han cargado los datos de nuestra base de datos (schema meteo).
2. **Limpieza JS:** se ha llevado a cabo la limpieza de los datos cargados por el paso anterior, donde se revisa si los datos están fuera de rango, que las temperaturas sean coherentes, que la humedad no sea negativa ni estén por encima de 100, etc.
3. **Filtrar filas nulas:** si encontramos filas con ciertos atributos nulos, se recogerán en otra tabla como inválidas.
4. **Lookup FK estacion:** verificaremos que las claves foráneas sean correctas. En caso de que alguna falle o sea errónea, se marcará como -1.
5. **Separar FK validas vs huerfanas:** separaremos los registros que queden representados con -1 del paso anterior en otra tabla.
6. **Ordenar:** ordenamos los datos en base a ciertos criterios como la fecha, la ciudad o el obs_id para facilitar la detección de datos duplicados.
7. **Unique Rows dedup:** se eliminan los datos duplicados manteniendo un registro por combinación de fecha y ciudad.
8. **Select Values:** seleccionamos solo las columnas relevantes para el modelo analítico.
9. **InsertUpdate staging.observaciones_clean:** insertamos los datos en la tabla “staging.observaciones_clean” con los datos limpios y sin duplicidad.
10. **Leer staging.observaciones_clean:** leemos los datos insertados en la etapa previa.
11. **InsertUpdate olap_snow.dim_clima:** finalmente, importamos los datos de la etapa anterior al sistema OLAP con los datos finales.

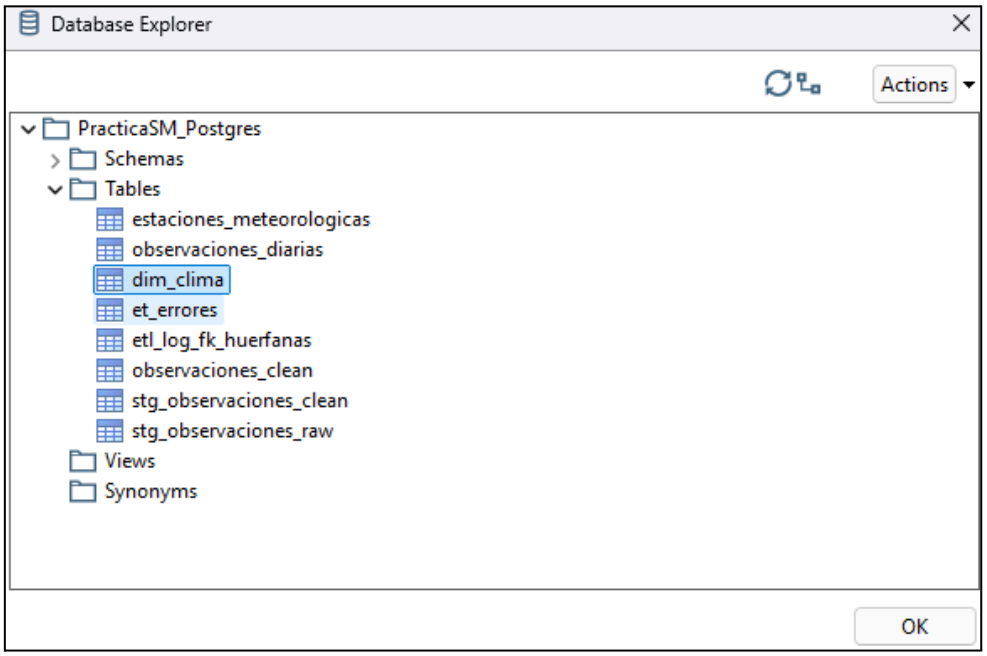
3.3. Resultados del flujo ETL

Cargamos el fichero **limpieza_meteo_completo.ktr** con cada uno de los pasos para realizar el ETL y lo ejecutamos.

Nota: para el correcto funcionamiento, se ha tenido que modificar el archivo de creación de staging y modificar la tabla **staging.etl_log_fk_huerfanas**.



Para ver los resultados, podemos hacer doble clic sobre nuestra conexion de la base de datos de **Pentaho > Explorar > Nombre_Conexion > Tables** y nos aparecerá **dim_clima**, que es la nueva dimensión:



Arriba en “**Actions**” podemos previsualizar los 100 primeros registros de la tabla:

Examine preview data

Rows of step: dim_clima (100 rows)

#	clima_sk	fecha	ciudad	anio	mes	dia_del_anio	temp_max	temp_min	temp_media	precipitacion_mm	humedad_pct	velocidad_viento_kmh	condicion_climatica	es_dia_lluvia	categoria_temperatura	fecha_carga
1	1	2020/01/01 00:00:00.000	Almería	2020	1	1	21.7	13.0	17.4	1.1	95	46.7	Niebla	<null>	<null>	2026/04/26 20:26:13.972207000
2	2	2020/01/01 00:00:00.000	Barcelona	2020	1	1	14.8	4.0	9.4	0.0	69	72.0	Niebla	<null>	<null>	2026/04/26 20:26:13.972207000
3	3	2020/01/01 00:00:00.000	Bilbao	2020	1	1	13.9	3.8	8.8	4.8	45	33.2	Tormenta	<null>	<null>	2026/04/26 20:26:13.972207000
4	4	2020/01/01 00:00:00.000	Granada	2020	1	1	14.5	6.2	10.3	5.4	38	47.6	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
5	5	2020/01/01 00:00:00.000	Madrid	2020	1	1	7.4	-2.2	2.6	2.9	78	14.2	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
6	6	2020/01/01 00:00:00.000	Málaga	2020	1	1	10.8	2.4	6.6	0.0	53	21.0	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
7	7	2020/01/01 00:00:00.000	Murcia	2020	1	1	11.7	2.0	6.8	12.4	33	25.2	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
8	8	2020/01/01 00:00:00.000	Sevilla	2020	1	1	15.5	9.0	12.2	0.4	37	49.6	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
9	9	2020/01/01 00:00:00.000	Valencia	2020	1	1	14.8	3.6	9.2	5.0	76	35.9	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
10	10	2020/01/01 00:00:00.000	Zaragoza	2020	1	1	7.6	-2.9	2.3	1.5	60	49.1	Parcialmente nublado	<null>	<null>	2026/04/26 20:26:13.972207000
11	11	2020/01/02 00:00:00.000	Almería	2020	1	2	16.9	8.0	12.4	2.0	91	11.2	Nublado	<null>	<null>	2026/04/26 20:26:13.972207000
12	12	2020/01/02 00:00:00.000	Barcelona	2020	1	2	11.5	2.1	6.8	9.1	46	59.4	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
13	13	2020/01/02 00:00:00.000	Bilbao	2020	1	2	15.2	2.9	9.0	0.3	51	68.4	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
14	14	2020/01/02 00:00:00.000	Granada	2020	1	2	10.8	-0.6	5.1	1.8	35	49.6	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
15	15	2020/01/02 00:00:00.000	Madrid	2020	1	2	4.1	-6.4	-1.2	1.5	36	31.5	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
16	16	2020/01/02 00:00:00.000	Málaga	2020	1	2	21.3	8.8	15.1	6.3	63	36.1	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
17	17	2020/01/02 00:00:00.000	Murcia	2020	1	2	21.6	13.3	17.5	9.3	74	70.6	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
18	18	2020/01/02 00:00:00.000	Sevilla	2020	1	2	20.3	11.4	15.9	3.5	41	25.5	Niebla	<null>	<null>	2026/04/26 20:26:13.972207000
19	19	2020/01/02 00:00:00.000	Valencia	2020	1	2	22.5	8.8	15.7	5.7	86	24.8	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
20	20	2020/01/02 00:00:00.000	Zaragoza	2020	1	2	12.5	3.0	7.8	6.5	62	69.8	Niebla	<null>	<null>	2026/04/26 20:26:13.972207000
21	21	2020/01/03 00:00:00.000	Almería	2020	1	3	12.0	5.5	8.8	0.2	35	21.5	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
22	22	2020/01/03 00:00:00.000	Barcelona	2020	1	3	16.6	5.8	11.2	0.1	41	52.9	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
23	23	2020/01/03 00:00:00.000	Bilbao	2020	1	3	13.6	4.1	8.8	2.5	70	65.3	Desconocido	<null>	<null>	2026/04/26 20:26:13.972207000
24	24	2020/01/03 00:00:00.000	Granada	2020	1	3	10.1	1.8	6.0	2.7	70	19.1	Nieve	<null>	<null>	2026/04/26 20:26:13.972207000
25	25	2020/01/03 00:00:00.000	Málaga	2020	1	3	14.9	4.7	9.8	0.0	40	50.8	Parcialmente nublado	<null>	<null>	2026/04/26 20:26:13.972207000

4. Implementación a nuestro sistema OLAP

En este apartado se ha desarrollado un flujo muy parecido al anterior, pero con varios cambios importantes:

- Los datos pertenecen a la tabla de hechos “**olap.fact_ventas**”, por lo que pueden no registrar aquellas ventas cuya FK “**id_fechas**” no estén registradas en la dimensión “**tiempo**”.

- Se han creado varios pasos 04 asociados a los Lookups del anterior flujo debido a las FKs de las ventas.

- Se ha añadido un paso adicional “**Select LOG huerfanas**” ya que las filas huérfanas no contenian la estructura correcta para ser almacenados en el “**staging.log_fk_huerfanas**”.

Para su implementación, se ha creado un schema “**raw**” en la base de datos con los **datos en bruto** en la tabla “**ventas_sucias**” y otro schema “**staging**” con los las tablas “**ventas_clean**” y “**log_fk_huerfanas**” con los datos recogidos durante el flujo ETL.

4.1. Análisis de los datos actuales

Para realizar el flujo, se han observado los siguientes errores en los datos localizados en nuestra tabla "**raw.ventas_sucias**":

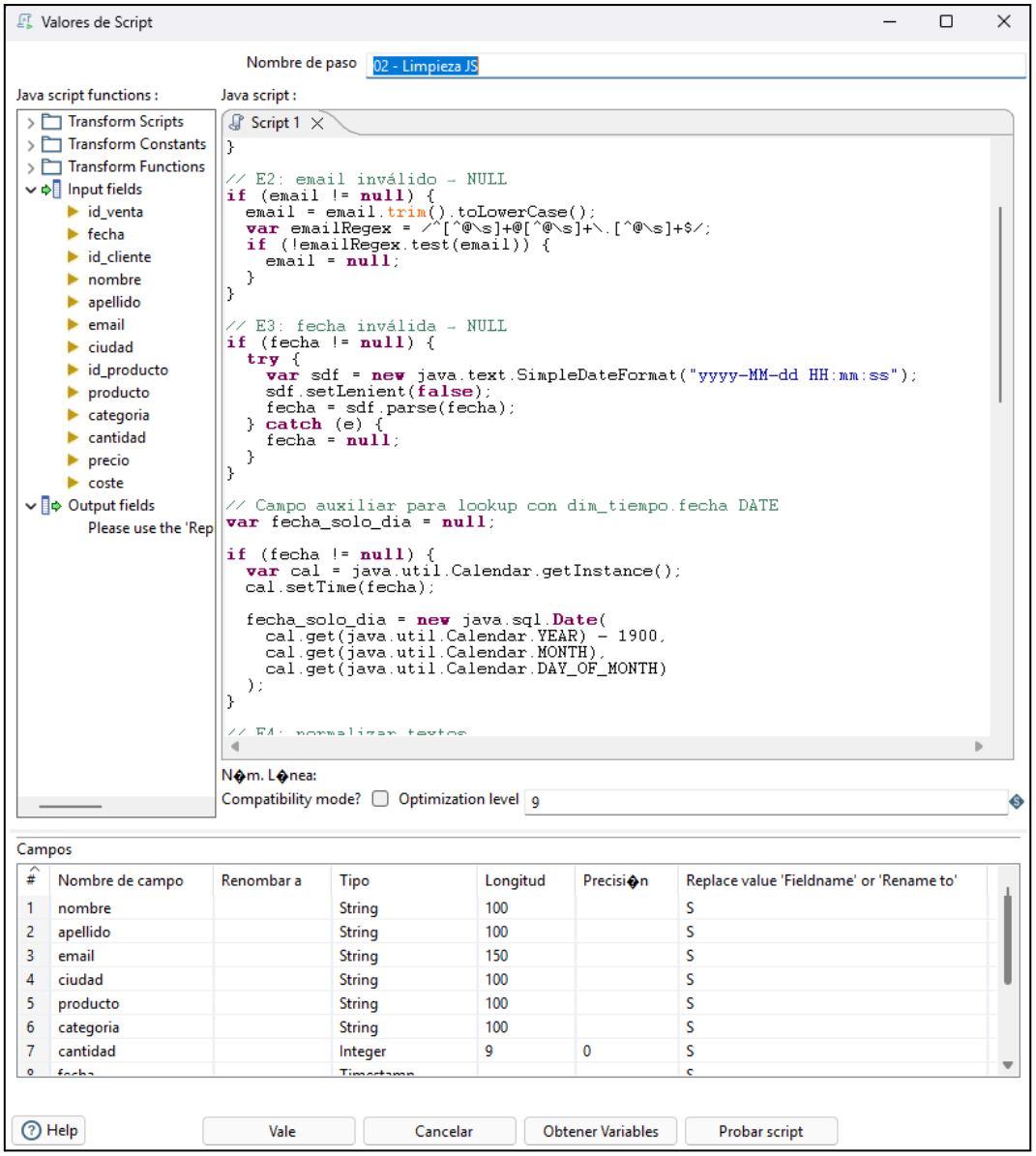
Problema	Ejemplo	Solución aplicada
Fecha mal formateada	21/03/2026'	Intentar parsear a yyyy-MM-dd HH:mm:ss. Si falla ⇒ NULL ⇒ se descarta.
Cantidad inválida	cantidad = 0	Si cantidad <= 0 ⇒ NULL ⇒ se descarta.
Campos vacíos	nombre = '', fecha = NULL	TRIM + convertir '' ⇒ NULL. Si falta campo obligatorio ⇒ descartar.
Categoría desconocida	categoria = '???'	Validar contra lista de categorías. Si no válida ⇒ NULL ⇒ descartar.
Precio mal formado	precio = 'abc'	Intentar conversión a número. Si falla ⇒ NULL ⇒ descartar.
Coste nulo	coste = NULL	Descartar (no se puede calcular beneficio correctamente).
Duplicado exacto	Misma fecha, cliente, producto, cantidad, precio y coste	Usar Sort + Unique Rows para eliminar duplicados por contenido.
FK cliente inexistente	id_cliente = 99 (no existe en dimensión)	Lookup ⇒ si no encuentra (-1) ⇒ enviar a LOG de huérfanas.
FK componente inexistente	id_componente = 99	Lookup ⇒ si no encuentra ⇒ LOG de huérfanas.
Fecha no existente en dimensión tiempo	fecha = '2026-04-01' (no está en dim_tiempo)	Lookup ⇒ si no encuentra ⇒ LOG de huérfanas.

4.2. Desarrollo del flujo

Como hemos mencionado antes, tenemos un flujo muy parecido al anterior. A continuación se detallarán los cambios realizados:

- **01 - Leer raw.ventas_sucias:** se ha cambiado la consulta para obtener los datos de la tabla "**raw.ventas_sucias**".

- **02 - Limpieza JS:** se ha editado el código de la limpieza de las tuplas:

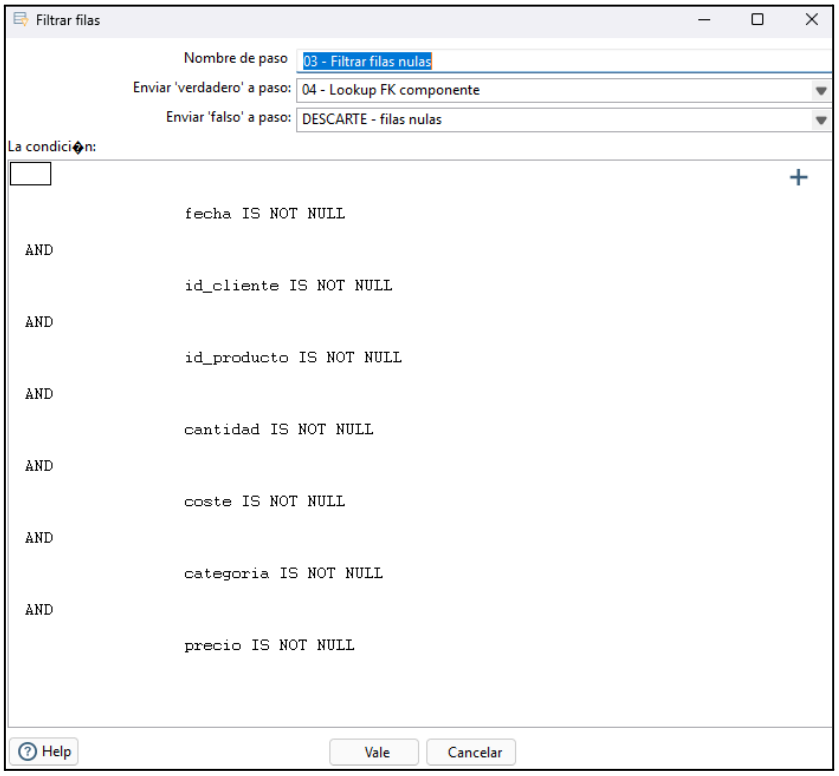


En esta tabla se recoge los problemas resueltos:

Bloque	Qué hace	Ejemplo
E1: nombres vacíos	Elimina espacios con trim() y convierte cadenas vacías en NULL.	Lucia ' → 'Lucia'; '' → NULL
E2: email inválido	Limpia el email, lo pasa a minúsculas y valida el formato con expresión regular. Si no es válido, lo pone a NULL.	LUIS_MAIL.COM' → NULL; 'ana@mail.com' válido
E3: fecha inválida	Intenta convertir la fecha usando el formato obligatorio yyyy-MM-dd HH:mm:ss. Si no encaja, la pone a NULL.	2026-03-05 09:15:00' válido; '21/03/2026' → NULL

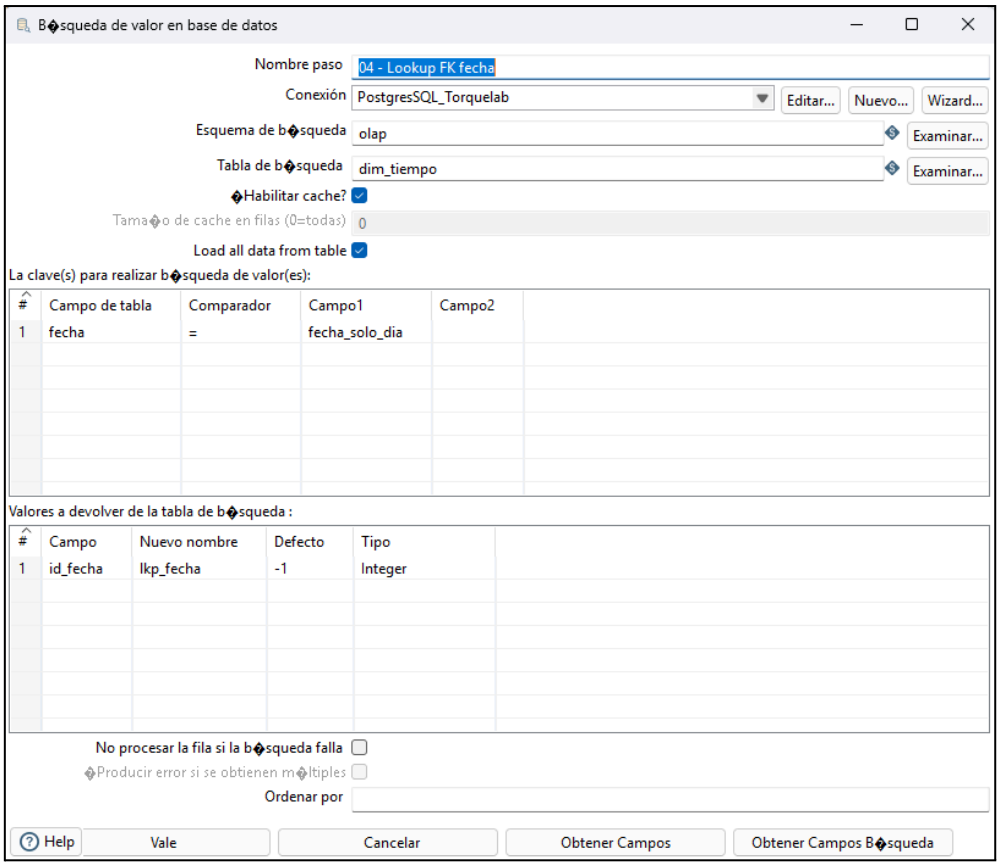
fecha_solo_dia	Crea una fecha auxiliar sin hora para hacer el lookup contra olap.dim_tiempo.fecha, que es tipo DATE.	2026-03-05 09:15:00' → '2026-03-05'
E4: normalizar textos	Normaliza ciudad y producto: quita espacios, pasa a minúsculas y capitaliza la primera letra.	MADRID ' → 'Madrid'; ' Frenos ' → 'Frenos'
E4: categoría	Normaliza categorías válidas. Si no reconoce la categoría, la pone a NULL para descartarla.	freno' → 'Frenos'; '???' → NULL
E5: cantidad inválida	Si la cantidad es menor o igual que 0, la convierte en NULL.	0 → NULL; -2 → NULL
E6: precio inválido	Convierte el precio a número. Cambia coma por punto. Si no se puede convertir, lo pone a NULL.	64.90' → 64.90; 'abc' → NULL
E7: coste inválido	Igual que precio, pero aplicado al coste.	34,00' → 34.00; NULL queda NULL
Importes totales	Multiplica precio y coste por la cantidad para obtener importes totales de la venta.	precio 37.80, cantidad 2 → 75.60
E8: beneficio	Calcula el beneficio como precio - coste.	75.60 - 38.00 = 37.60

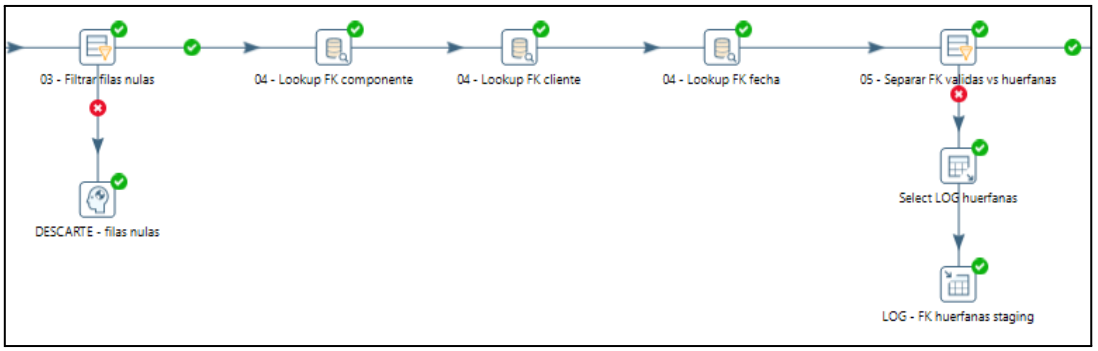
- **03 - Filtrar filas nulas:** se ha cambiado los valores a “**fecha**”, “**id_cliente**”, “**id_producto**”, “**cantidad**”, “**coste**”, “**categoria**” y “**precio**” como valores imprescindibles que una tupla limpia debe tener como mínimo:



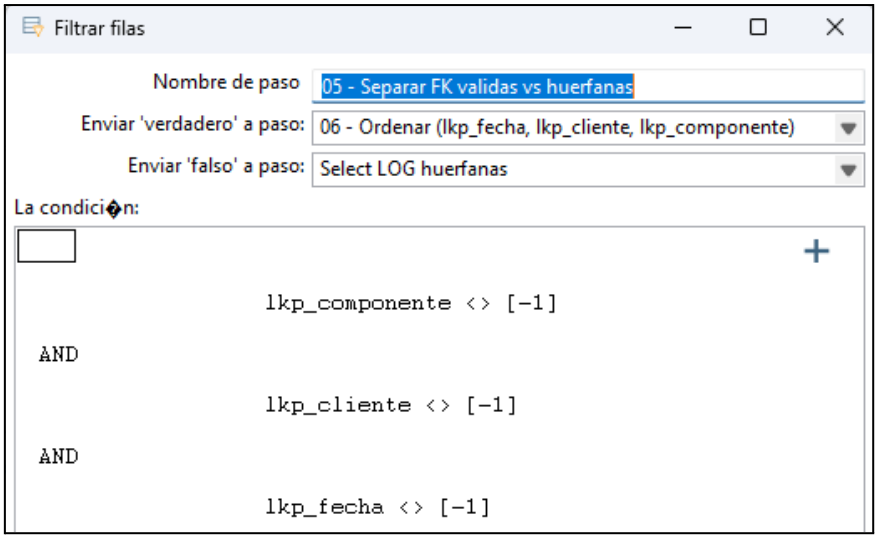
- **DESCARTE - filas nulas:** se ha ajustado la tabla destino.

- **04 - Lookup FK componente / 04-Lookup FK cliente /04-Lookup FK fecha :** como tenemos tres claves foráneas, se ha añadido 3 pasos secuenciales. Comprobará los datos del sistema OLAP de las tablas "**dim_cliente**", "**dim_componente**" y "**dim_tiempo**":

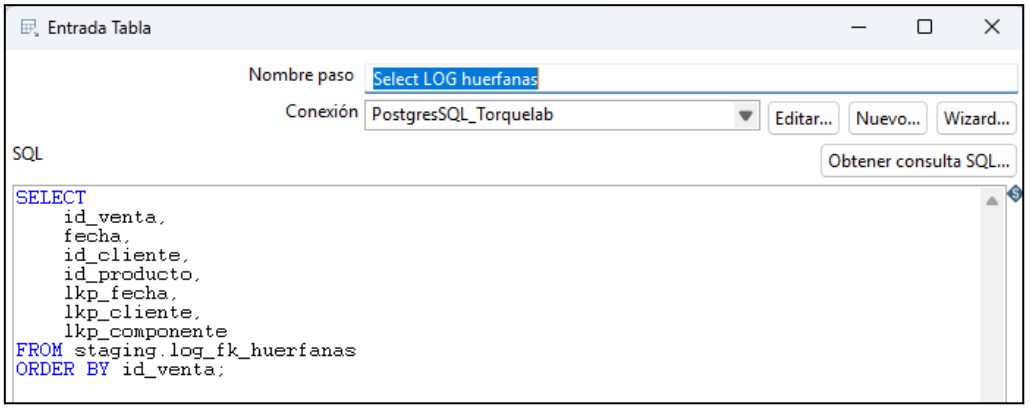




- **05 - Separar FK validas vs huerfanas:** se ha ajustado a nuestras claves:

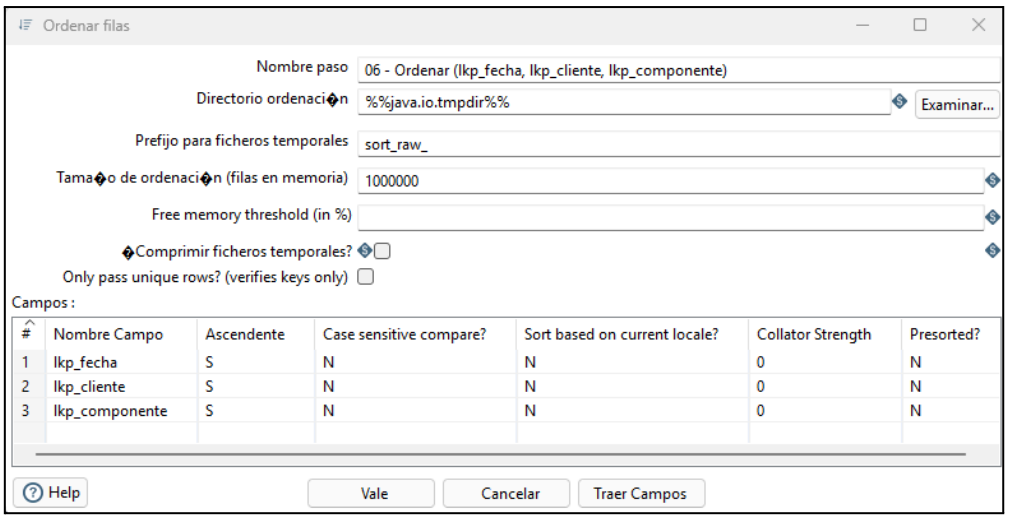


- **Select LOG huerfanas:** se ha añadido este paso para conseguir la estructura correspondiente para su almacenamiento puesto que se está intentando guardar un campo "fecha" que al principio se recoge como Timestamp, pero debe ser de tipo Date.

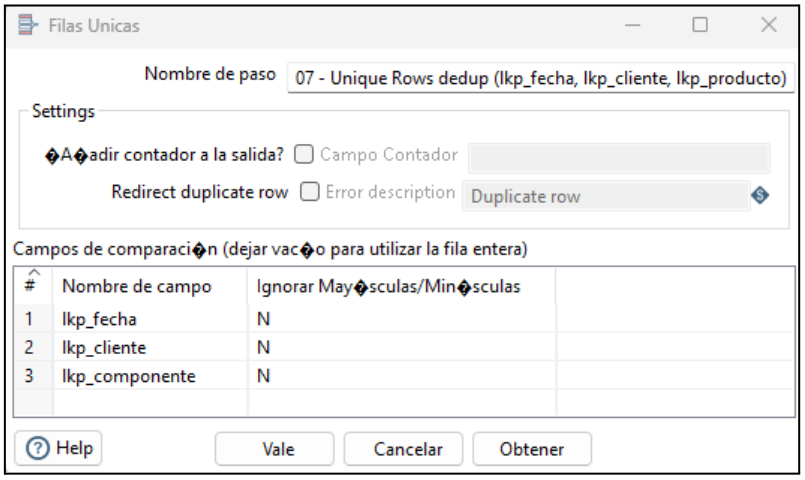


- **LOG - FK huerfanas staging:** se ha ajustado la tabla destino.

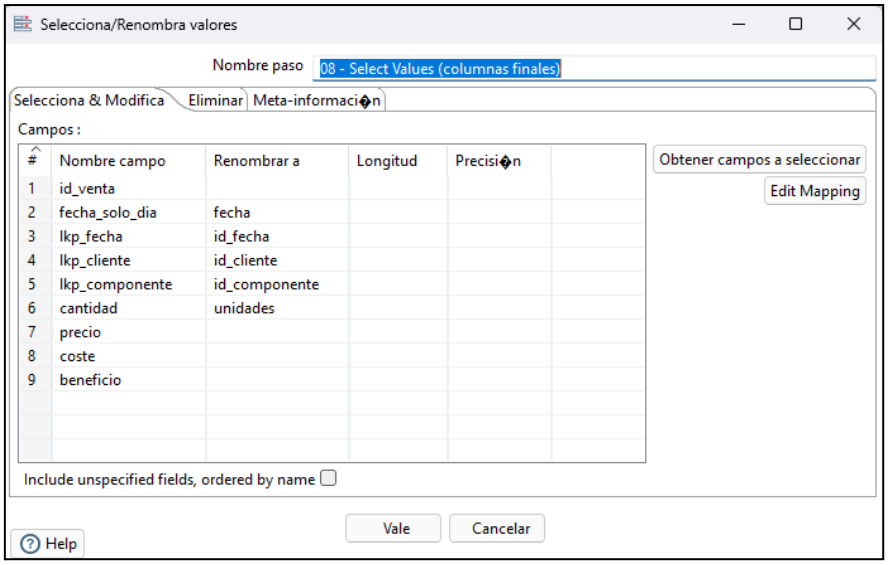
- **06 - Ordenar (lkp_fecha, lkp_cliente, lkp_componente):** se ha ajustado el orden. Si coincide "lkp_fecha" junto al "lkp_cliente" y el "lkp_componente", se considerará duplicado en el paso siguiente:



- **07 - Unique Rows dedup (lkp_fecha, lkp_cliente, lkp_producto):** se ha ajustado para eliminar los duplicados del paso anterior:

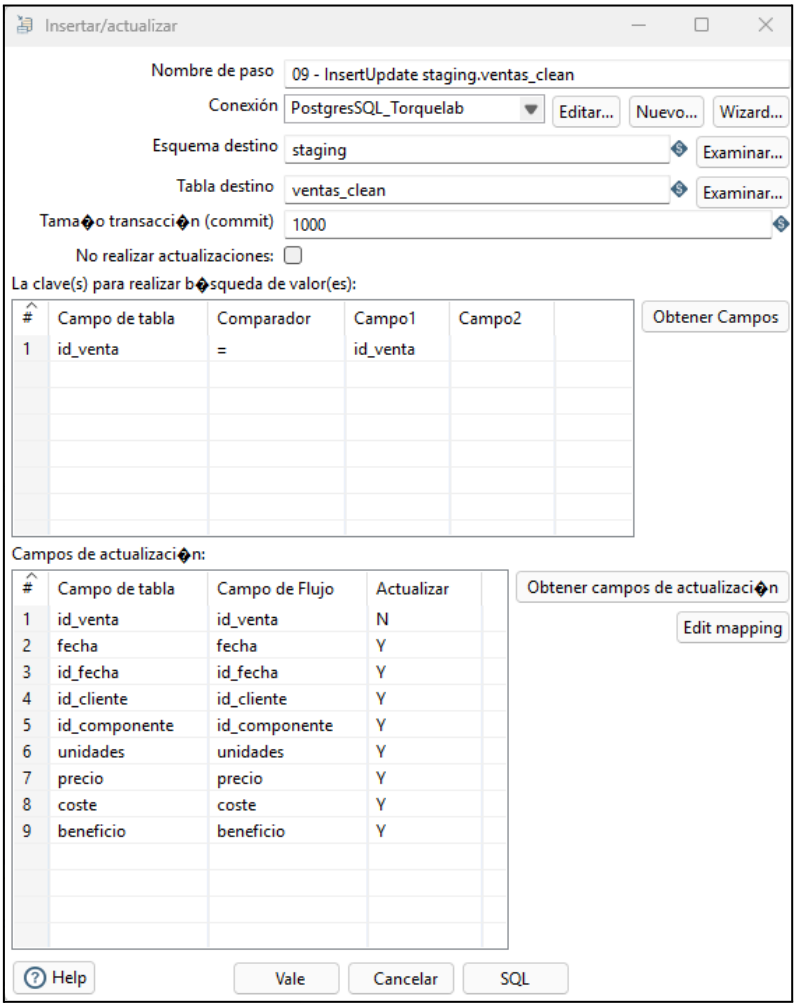


- **08 - Select Values (columnas finales):** se han ajustado los valores:



#	Nombre campo	Renombrar a	Longitud	Precisi⌂n
1	id_venta			
2	fecha_solo_dia	fecha		
3	lkp_fecha	id_fecha		
4	lkp_cliente	id_cliente		
5	lkp_componente	id_componente		
6	cantidad	unidades		
7	precio			
8	coste			
9	beneficio			

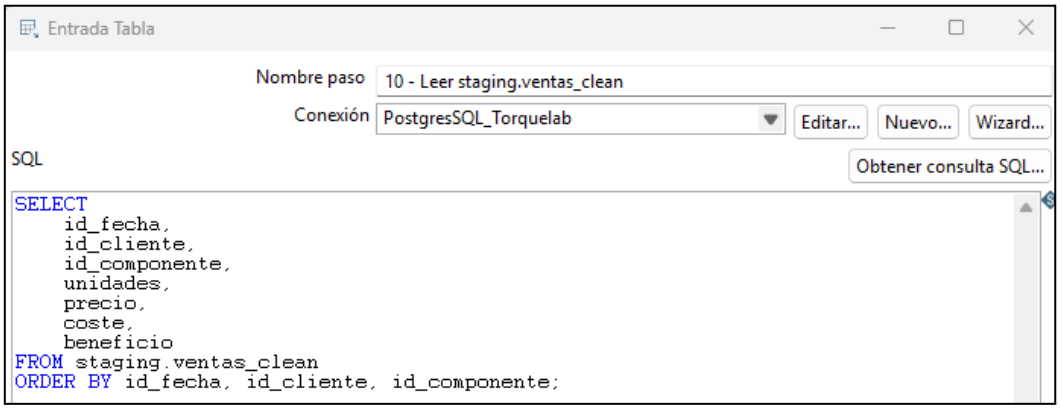
- **09 - InsertUpdate staging.ventas_clean:** se han ajustado los campos, y la clave a “id_venta”:



#	Campo de tabla	Comparador	Campo1	Campo2
1	id_venta	=	id_venta	

#	Campo de tabla	Campo de Flujo	Actualizar
1	id_venta	id_venta	N
2	fecha	fecha	Y
3	id_fecha	id_fecha	Y
4	id_cliente	id_cliente	Y
5	id_componente	id_componente	Y
6	unidades	unidades	Y
7	precio	precio	Y
8	coste	coste	Y
9	beneficio	beneficio	Y

- **10 - Leer staging.ventas_clean:** Se ha cambiado la consulta para leer los datos limpios y estructurados:



- **11 - InsertUpdate olap.fact_ventas:** Se ha ajustado la tabla destino y los valores de las claves y campos:

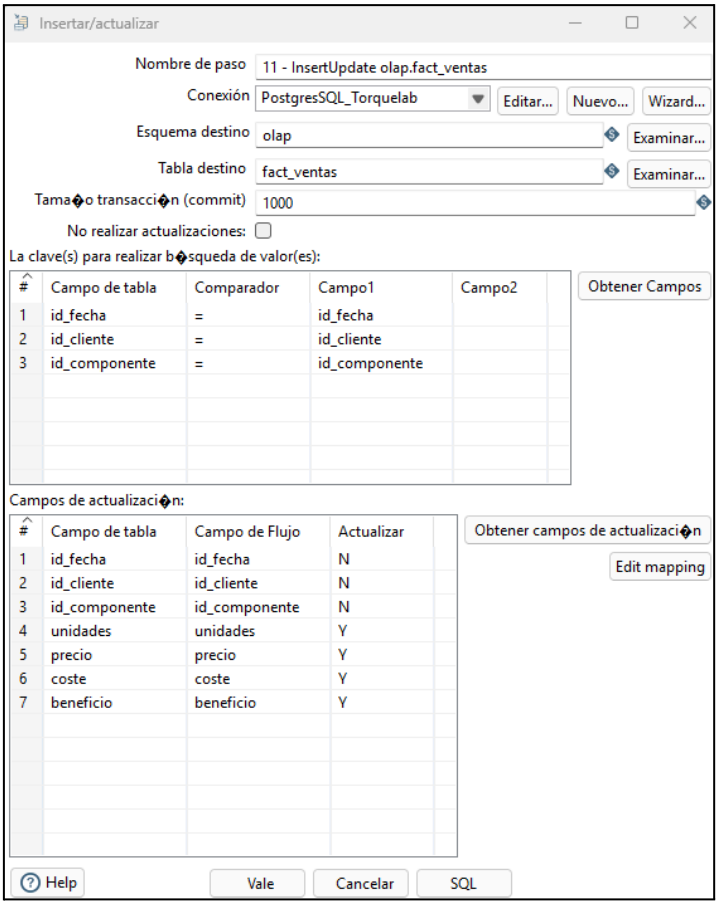
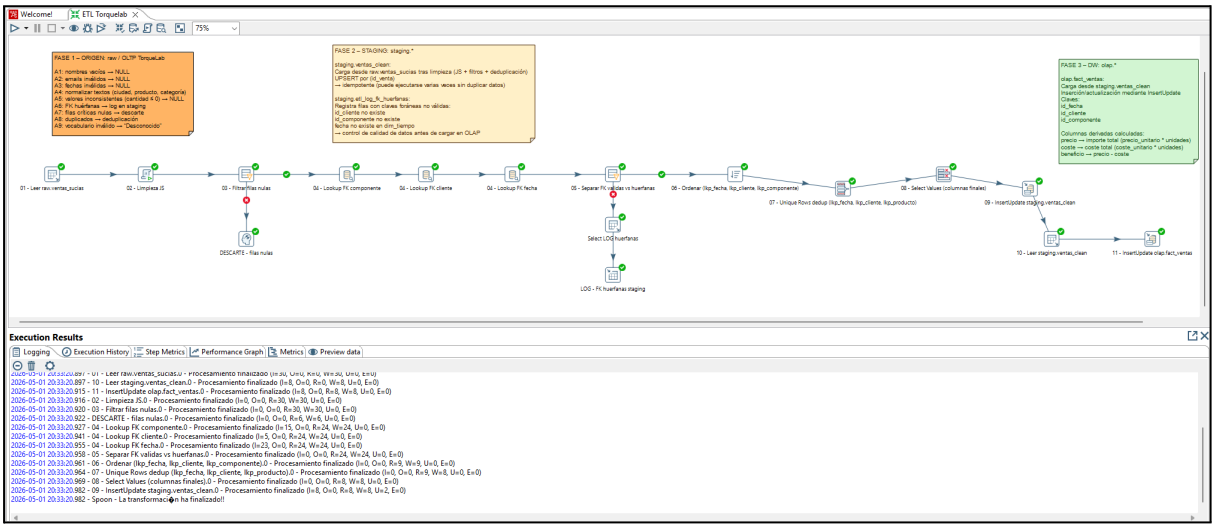


Tabla resúmen:

Fichero / paso	Propósito	Entrada	Salida
01 - Leer raw.ventas_sucias	Leer datos sucios de ventas	raw.ventas_sucias	Flujo inicial
02 - Limpieza JS	Limpiar fechas, textos, emails, cantidades, precios, costes y calcular beneficio	Datos raw	Datos normalizados
03 - Filtrar filas nulas	Separar registros válidos de registros con campos obligatorios nulos	Datos limpios	Filas válidas / descartadas
DESCARTE - filas nulas	Recoger filas descartadas por errores de limpieza	Rama falsa del filtro	Registros descartados
04 - Lookup FK componente	Buscar el componente en olap.dim_componente	id_producto	lkp_componente
04 - Lookup FK cliente	Buscar el cliente en olap.dim_cliente	id_cliente	lkp_cliente
04 - Lookup FK fecha	Buscar la fecha en olap.dim_tiempo	fecha_solo_dia	lkp_fecha
05 - Separar FK validas vs huerfanas	Separar registros con FK válidas de registros huérfanos	Datos con lookups	Rama válida / rama huérfana
Select LOG huerfanas	Adaptar columnas para guardar el log de FK huérfanas	Rama huérfana	Campos preparados para log
LOG - FK huerfanas staging	Registrar errores de FK	Datos huérfanos preparados	Tabla LOG en staging
06 - Ordenar (lkp_fecha, lkp_cliente, lkp_componente)	Ordenar registros válidos antes de deduplicar	Rama válida	Datos ordenados
07 - Unique Rows dedup (lkp_fecha, lkp_cliente, lkp_producto)	Eliminar duplicados	Datos ordenados	Datos únicos
08 - Select Values (columnas finales)	Seleccionar y renombrar columnas finales	Datos únicos	Campos listos para staging
09 - InsertUpdate staging.ventas_clean	Insertar/actualizar ventas limpias en staging	Campos finales	staging.ventas_clean
10 - Leer staging.ventas_clean	Leer ventas limpias desde staging	staging.ventas_clean	Flujo para OLAP
11 - InsertUpdate olap.fact_ventas	Insertar/actualizar la tabla de hechos	Datos de staging	olap.fact_ventas

4.4. Visualización de resultados



Tras realizar el flujo completo, nos encontramos con los siguientes datos en la base de datos:

- **staging.ventas_clean:** Tabla vacía en un inicio.

Showing rows: 1 to 8 | Page No: 1 of 1

	id_venta integer	fecha date	id_fecha bigint	id_cliente bigint	id_componente bigint	unidades integer	precio numeric (12,2)	coste numeric (12,2)	beneficio numeric (12,2)
1	118	2026-03-05	2	4	7	1	199.00	132.00	67.00
2	102	2026-03-07	3	3	3	1	119.00	71.00	48.00
3	119	2026-03-07	3	5	8	1	245.00	160.00	85.00
4	104	2026-03-09	5	5	5	1	599.00	410.00	189.00
5	116	2026-03-21	17	2	5	1	599.00	410.00	189.00
6	117	2026-03-22	18	3	6	1	289.00	190.00	99.00
7	101	2026-03-05	2	2	2	1	64.90	34.00	30.90
8	103	2026-03-08	4	4	4	1	129.90	82.00	47.90

- **olap.fact_ventas:** Tabla con 15 tuplas en un inicio.

	id_fact_venta [PK] bigint	id_fecha bigint	id_cliente bigint	id_componente bigint	unidades integer	precio numeric (12,2)	coste numeric (12,2)	beneficio numeric (12,2)
6	6	4	7	6	1	289.00	190.00	99.00
7	7	7	8	7	1	199.00	132.00	67.00
8	8	10	9	8	1	245.00	160.00	85.00
9	9	13	10	9	1	49.90	24.00	25.90
10	10	17	11	10	1	89.90	51.00	38.90
11	11	1	12	11	1	24.00	12.00	12.00
12	12	3	13	12	1	68.00	34.00	34.00
13	13	7	14	13	1	140.00	84.00	56.00
14	14	11	15	14	1	158.00	79.00	79.00
15	15	15	16	15	1	820.00	410.00	410.00
16	16	2	2	2	1	64.90	34.00	30.90
17	17	2	4	7	1	199.00	132.00	67.00
18	18	3	3	3	1	119.00	71.00	48.00
19	19	3	5	8	1	245.00	160.00	85.00
20	20	4	4	4	1	129.90	82.00	47.90
21	21	5	5	5	1	599.00	410.00	189.00
22	22	17	2	5	1	599.00	410.00	189.00
23	23	18	3	6	1	289.00	190.00	99.00

Total rows: 23 | Query complete 00:00:00.061 | CRLF

4.5. Decisiones de diseño

Debido a las inconsistencias de los datos iniciales, se ha optado por realizar un ETL en lugar de hacer un ELT, puesto que proporciona un mejor control en el proceso de limpieza y una mejor trazabilidad que el hecho de aplicar la transformación directamente sobre el SGBD.

El proceso implementado es de tipo full load, ya que procesamos todos los datos disponibles en cada etapa del flujo. Así simplificamos el diseño del ETL y evitamos la complejidad de las cargas incrementales. Además, el número de registros de nuestra base de datos no supone un coste demasiado elevado para que cargarlo cada vez que hacemos el proceso de ETL suponga un problema.

5. Integración de objetivos en consulta analítica

Se quiere especificar y llevar a cabo la trazabilidad de los objetivos en las ventas de los productos de la empresa por categoría. Para ello, realizamos

5.1. Fichero CSV

Los datos contenidos en el fichero son los siguientes:

```
categoria;objetivo_ingresos
Motor;12000
Frenos;9000
Suspensión;15000
Electrónica;8000
Escape;10000
```

5.2. Diseño del flujo

El diseño del flujo es el siguiente:

1. **Extracción de datos:** Cogeremos los datos del fichero CSV anterior (**objetivos_categorias.csv**) y de la tabla "**olap.fact_ventas**".
2. **Unión de datos:** Posteriormente ordenaremos los datos y los uniremos según la clave "categoria".
3. **Gestión de datos:** Crearemos la constante 100 y calcularemos el porcentaje de margen de beneficios sobre las ventas y el porcentaje del objetivo. Posteriormente lo ordenaremos en ingresos descendentes.
4. **Almacenamiento de datos:** Seleccionamos los datos que necesitemos y lo pasaremos a una tabla en nuestra base de datos (**staging.resultados_analisis**) y a un CSV (**resultados_objetivos.csv**).

La tabla con los pasos detallados es la siguiente:

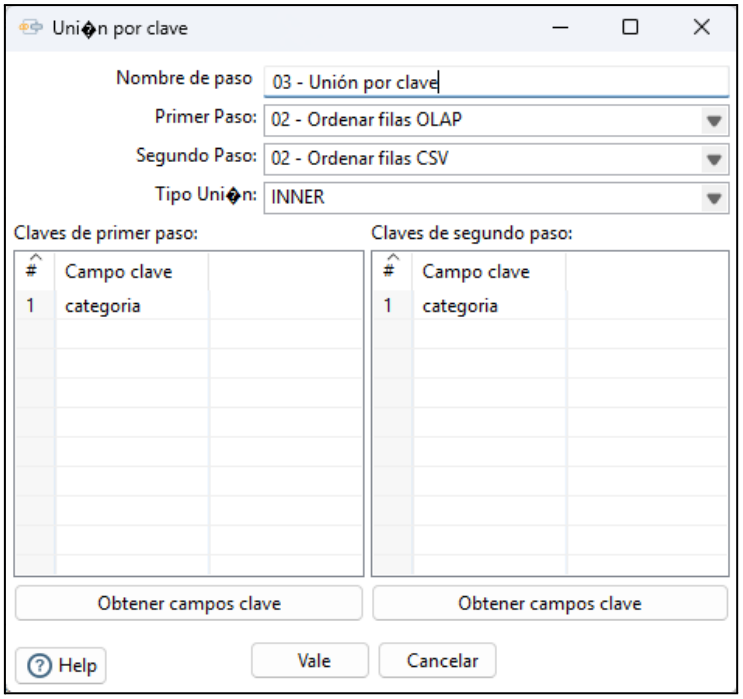
Fichero / paso	Propósito	Entrada	Salida
01 - Entrada Tabla	Leer datos agregados desde olap.fact_ventas	Consulta SQL (categoria, ingresos, costes, beneficios)	Flujo OLAP
02 - Ordenar filas OLAP	Ordenar por categoría	Flujo OLAP	Flujo OLAP ordenado
01 - Entrada CSV	Leer fichero de objetivos	Fichero CSV (categoria, objetivo_ingresos)	Flujo CSV

02 - Ordenar filas CSV	Ordenar por categoría	Flujo CSV	Flujo CSV ordenado
03 - Unión por clave	Unir OLAP y CSV	Flujo OLAP ordenado + Flujo CSV ordenado	Flujo combinado
04 - Constante 100	Añadir valor fijo	Flujo combinado	Flujo con constante
05 - Calcular porcentajes	Calcular métricas	Flujo con constante	Flujo con pct_margen y pct_objetivo_cumplido
06 - Ordenar ingresos DESC	Ordenar resultados	Flujo calculado	Flujo ordenado final
07 - Seleccionar/Renombrar valores	Limpiar columnas	Flujo ordenado	Flujo final
08 - Insertar / Actualizar	Guardar en BD	Flujo final	Tabla staging.resultados_analisis
09 - Salida Fichero CSV	Exportar resultados	Flujo final	Fichero CSV

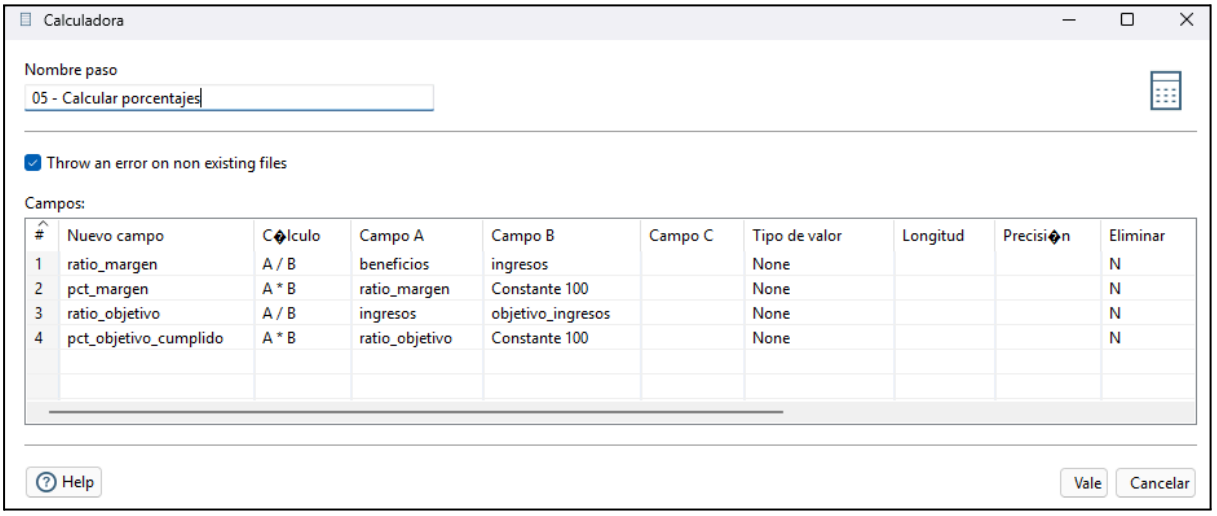
5.3. Configuración de los pasos

En este apartado se mostrará la configuración de alguno de los pasos más destacados:

- **03 - Unión por clave:**



- **05 - Calcular porcentajes:**



#	Nuevo campo	C⌂lculo	Campo A	Campo B	Campo C	Tipo de valor	Longitud	Precisi⌂n	Eliminar
1	ratio_margen	A / B	beneficios	ingresos		None			N
2	pct_margen	A * B	ratio_margen	Constante 100		None			N
3	ratio_objetivo	A / B	ingresos	objetivo_ingresos		None			N
4	pct_objetivo_cumplido	A * B	ratio_objetivo	Constante 100		None			N

- **08 - Insertar / Actualizar:**

Insertar/actualizar

Nombre de paso: 08 - Insertar / Actualizar

Conexión: PostgresSQL_Torquelab | Editar... | Nuevo... | Wizard...

Esquema destino: staging | Examinar...

Tabla destino: resultados_analisis | Examinar...

Tama$\hat{o}$o transacci$\hat{o}$n (commit): 100

No realizar actualizaciones: ☐

La clave(s) para realizar b$\hat{o}$squeda de valor(es):

#	Campo de tabla	Comparador	Campo1	Campo2
1	categoria	=	categoria	

Obtener Campos

Campos de actualizaci$\hat{o}$n:

#	Campo de tabla	Campo de Flujo	Actualizar
1	categoria	categoria	Y
2	ingresos	ingresos	Y
3	costes	costes	Y
4	beneficios	beneficios	Y
5	objetivo_ingresos	objetivo_ingresos	Y
6	pct_margen	pct_margen	Y
7	pct_objetivo_cumplido	pct_objetivo_cumplido	Y

Obtener campos de actualizaci$\hat{o}$n

Edit mapping

Help | Vale | Cancelar | SQL

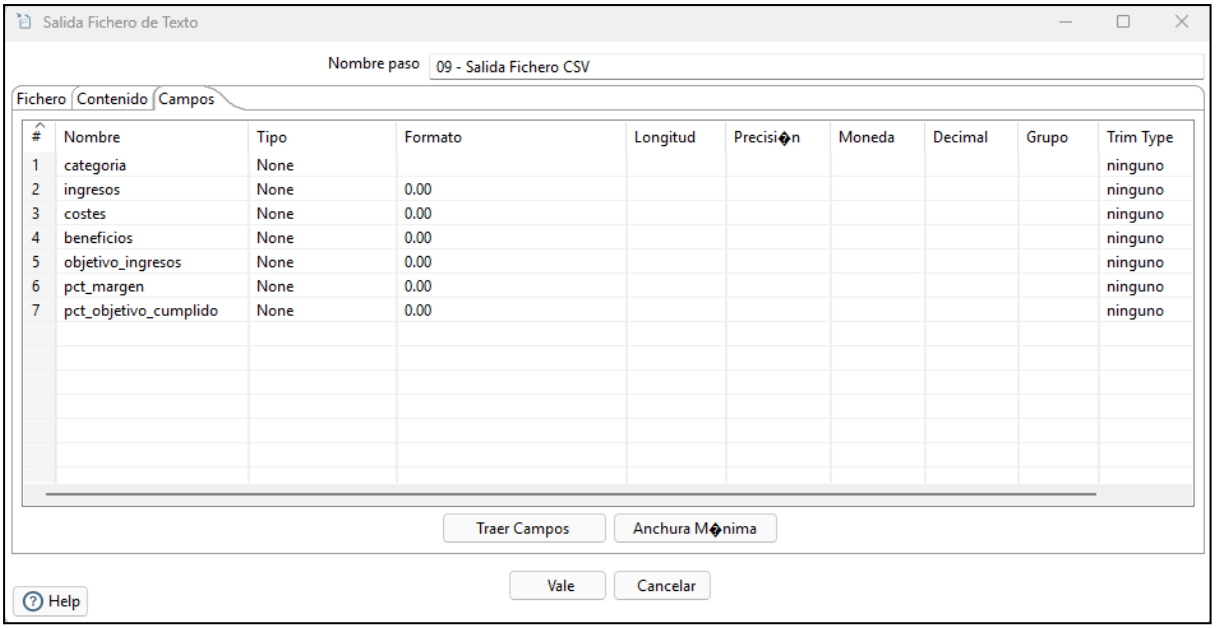
Simple SQL editor

SQL statements, separated by semicolon ';'

```
CREATE TABLE IF NOT EXISTS staging.resultados_analisis (
    categoria VARCHAR(100),
    ingresos NUMERIC(12,2),
    costes NUMERIC(12,2),
    beneficios NUMERIC(12,2),
    objetivo_ingresos NUMERIC(12,2),
    pct_margen NUMERIC(12,2),
    pct_objetivo_cumplido NUMERIC(12,2)
);

CREATE INDEX IF NOT EXISTS idx_resultados_analisis_lookup
ON staging.resultados_analisis(categoria);
```

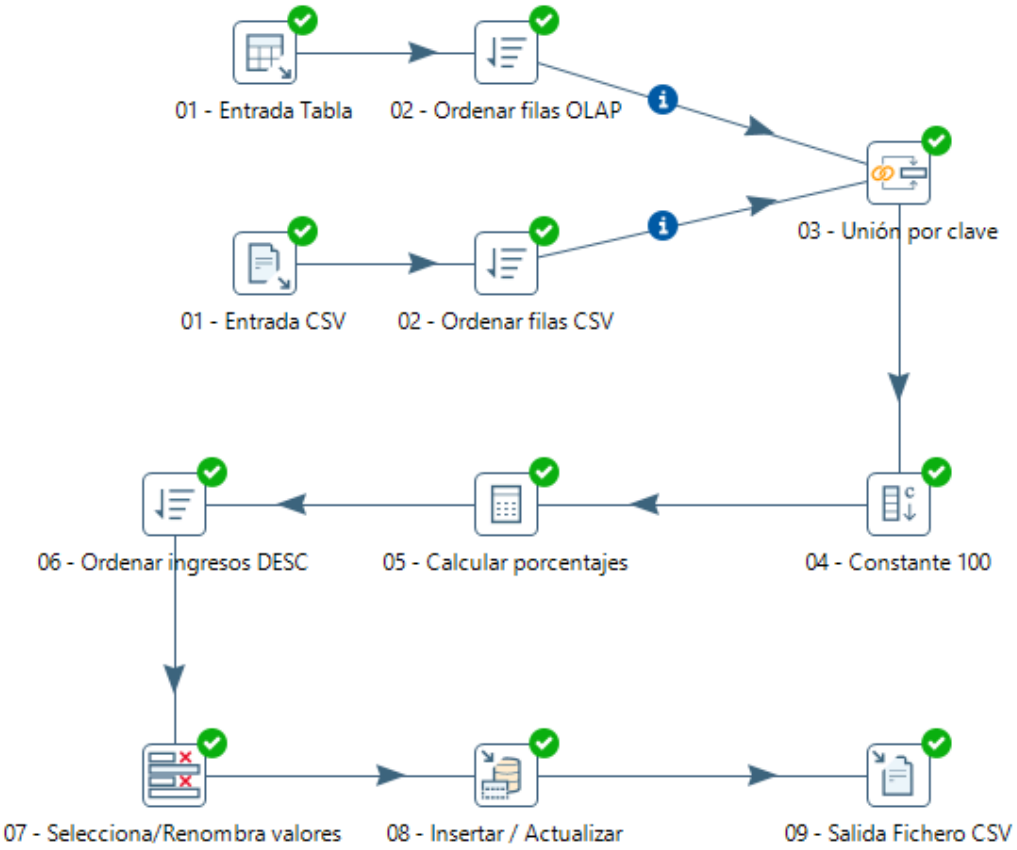
- **09 - Salida Fichero CSV:**



#	Nombre	Tipo	Formato	Longitud	Precisi n	Moneda	Decimal	Grupo	Trim Type
1	categoria	None							ninguno
2	ingresos	None	0.00						ninguno
3	costes	None	0.00						ninguno
4	beneficios	None	0.00						ninguno
5	objetivo_ingresos	None	0.00						ninguno
6	pct_margen	None	0.00						ninguno
7	pct_objetivo_cumplido	None	0.00						ninguno

5.4. Resultados

Nuestro diseño final en Pentaho es el siguiente:



Y los datos de los resultados son:

- **Fichero CSV:**

resultados_objetivos.csv

```
categoria;ingresos;costes;beneficios;objetivo_ingresos;pct_margen;pct_objetivo_cumplido
Escape;2706.90;1691.00;1015.90;10000.00;37.53;27.07
Suspensión;868.00;546.00;322.00;15000.00;37.10;5.79
Motor;639.80;411.00;228.80;12000.00;35.76;5.33
Frenos;595.80;366.00;229.80;9000.00;38.57;6.62
Electrónica;467.70;267.00;200.70;8000.00;42.91;5.85
```

- **Tabla staging.resultados_analisis:**

No limit

Query | Query History | AI Assistant | Scratch Pad

```
SELECT * FROM staging.resultados_analisis
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	categoria character varying (	ingresos double pre	costes double prec	beneficios double preci	objetivo_ingresos double precision	pct_margen double precision	pct_objetivo_cumplido double precision
1	Escape	2706.9	1691	1015.9	10000	37.53001588533008	27.069
2	Suspensión	868	546	322	15000	37.096774193548384	5.786666666666666
3	Motor	639.8	411	228.8	12000	35.761175367302286	5.331666666666666
4	Frenos	595.8	366	229.8	9000	38.56998992950655	6.619999999999999
5	Electrónica	467.7	267	200.7	8000	42.912123155869146	5.84625